

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 2 – מעודכן לתאריך 21.06.2026
עמוד 1 מתוך 10



מתרגל ממונה על התרגיל: יצחק גרינבוים, Yitzchakg@campus.technion.ac.il

תאריך ושעת הגשה: 19/07/2026 בשעה 23:59

אופן ההגשה: בזוגות. אין להגיש ביחידים. (אלא באישור מתרגל אחראי של הקורס)

הנחיות כלליות:

- שאלות על התרגיל יש לפרסם באתר הפיאצה של הקורס תחת לשונית "hw-wet-2":
 - האתר: <https://piazza.com/technion.ac.il/spring2026/234218>, נא לקרוא את השאלות של סטודנטים אחרים לפני שמפרסמים שאלה חדשה, למקרה שנשאלה כבר.
- נא לקרוא את המסמך "נהלי הקורס" באתר הקורס. בנוסף, נא לקרוא בעיון את כל ההנחיות בסוף מסמך זה.
- בפורום הפיאצה ינוהל FAQ ובמידת הצורך יועלו תיקונים כהודעות נעוצות (Pinned Notes). תיקונים אלו מחייבים.
- התרגיל מורכב משני חלקים: יבש ורטוב.
 - לאחר קריאת כלל הדרישות, מומלץ לתכנן תחילה את מבני הנתונים על נייר. דבר זה יכול לחסוך לכם זמן רב.
 - לפני שאתם ניגשים לקודד את פתרוןכם, ודאו כי יש לכם פתרון העומד בכל דרישות הסיבוכיות בתרגיל. תרגיל שאינו עומד בדרישות הסיבוכיות יחשב כפסול.
 - את הפתרון שלכם מומלץ לחלק למחלקות שונות שאפשר לממש (ולבדוק!) בהדרגתיות.
 - המלצות לפתרון התרגיל נמצאות באתר הקורס תחת: "Programming Tips Session".
- ההמלצות לתכנות במסמך זה אינן מחייבות, אך מומלץ להיעזר בהן.
- חומר התרגיל הינו כל החומר שנלמד בהרצאות ובתרגולים עד מיונים (לא כולל).
 - העתקת תרגילי בית רטובים תיבדק באמצעות תוכנת בדיקות אוטומטית, המזהה דמיון בין כל העבודות הקיימות במערכת, גם כאלו משנים קודמות. לא ניתן לערער על החלטת התוכנה. התוכנה אינה מבדילה בין מקור להעתק! אנא הימנעו מהסתכלות בקוד שאינו שלכם.
- בקשות להגשה מאוחרת יש להפנות למתרגל האחראי בלבד: goldstein@campus.technion.ac.il.
- בהצלחה!



הקדמה:

הכל התחיל לקראת ההאקתון הפקולטי השנתי. סגל הקורס 'מבני נתונים' ישב וחשב: "5,000 שקלים זה נחמד, אבל זה לא יגרום לסטודנטים שלנו באמת להזיע. לאחר סדרת דיונים ליליים מורטת עצבים, פגישות דחופות עם הדיקן וסגן הדיקן של הפקולטה, והפעלת לחץ מתוחכם (שכלל הבטחה למגן במבני נתונים) – הושג הבלתי יאומן: מימון של מיליון שקלים שלמים למנצחים בתחרות "המירוץ למיליון" הפקולטית! השמועה פשטה במסדרונות כמו אש בשדה קוצים. מאות סטודנטים עזבו את LeetCode, סגרו את chatgpt והחלו להתאגד לקבוצות במטרה לזכות בפרס הגדול ולכסות את שכר הלימוד (ולהחזיר את פינת הקפה בחווה).

כדי לנהל את הטירוף הזה, סגל הפקולטה הבין שצריך מערכת ניטור חזקה בזמן אמת. לשם כך, שקד המתרגל האחראי וכלבו הנאמן גורי שכרו את שירותכם (תמורת 3 אחוז מהציון הסופי) על מנת ליצור מערכת שעוקבת אחר התקדמות הקבוצות המשתתפות במירוץ. שקד וגורי, שמכירים את היכולות שלכם כמתכנתי עילית ויודעים שפלטפורמות כמו ChatGpt נוטות לקרוס ולא מסוגלות ליצור את המערכת המורכבת, מבקשים את עזרתכם בפיתוח המערכת החכמה: "Racenion".



חשוב להבהיר: התחרות אינה לתוכנית הטלוויזיה הידועה. בגרסה הפקולטית, המשתתפים מתאגדים לקבוצות, מתחרים במשימות שונות,

צוברים ניסיון, ולעיתים אף מצליחים לגייס קבוצות אחרות להצטרף אליהם לפי חוקי התחרות.

חוקים והנחות של המערכת

1. מתמודד לא עוזב קבוצה שהוא שייך אליה.
2. כשקבוצה מודחת, היא לא יכולה להשתתף במשימות, אבל המידע על המתמודדים שלה נשאר במערכת.
3. המתמודדים בקבוצה מסודרים לפי הסדר הכרונולוגי בו הצטרפו.
4. לכל מתמודד יש מיומנות (Skill), שהיא מיוצגת כמטריצה 2×2 הפיכה ב $GL(2, \mathbb{F}_p)$ עם p ראשוני כלשהו (אין צורך להכיר), ומוטיבציה המיוצגת כשלם אי שלילי.
5. לכל קבוצה יש מיומנות משותפת, שהיא מכפלת המיומנויות (כפל מטריצות) של חבריה לפי הסדר של ההצטרפות (הסבר בנקודות למטה), וגם ניסיון (מספר שלם).
6. קבוצה יכולה לגייס אליה קבוצה אחרת במידה שתנאים מסוימים מתקיימים (מפורט היכן שרלוונטי). במקרה כזה, כל המתמודדים שהיו בקבוצה המגייסת ייחשבו קודמים בסידור הכרונולוגי של ההצטרפות לקבוצה. במפורש, אם A מגייסת את B אליה:

$$A = (a_1, \dots, a_{|A|}), \quad B = (b_1, \dots, b_{|B|}) \Rightarrow A \text{ recruits } B \text{ yields } (a_1, \dots, a_{|A|}, b_1, \dots, b_{|B|})$$

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 2 – מעודכן לתאריך 21.06.2026
עמוד 3 מתוך 10



נקודות חשובות

- סיפקנו לכם את המחלקה Skill, עם פעולות יצירה, העתקה, השמה, בדיקת תקינות, איבר אדיש כפלי, פעולות אריתמטיות ($*$, $=$), החזרת ההופכית, החזרת מיומנות אפקטיבית, אופרטור השוואה ($=$, $<$, $>$), דטרמיננטה והדפסה. אין צורך לממש את המחלקה הזו בעצמכם.
- בכל אחת מהפונקציות שזמן הריצה שלה משוערך פרט לפונקציה add_contestant, השערוך הוא יחד עם כל הפונקציות האחרות פרט לפונקציה add_contestant.

דרוש מבנה נתונים למימוש הפעולות הבאות:

Racenion()

מאתחלת מבנה נתונים ריק. תחילה אין במערכת קבוצות או מתמודדים.

פרמטרים: אין.

ערך החזרה: אין.

סיבוכיות זמן: $O(1)$ במקרה הגרוע.

virtual ~Racenion()

הפעולה משחררת את המבנה (כל הזיכרון אותו הקצתם חייב להיות משוחרר).

פרמטרים: אין.

ערך החזרה: אין.

סיבוכיות זמן: $O(n + k)$ במקרה הגרוע, כאשר n הוא מספר המתמודדים הכולל במערכת ו- k הוא מספר הקבוצות מאז תחילת המערכת.

StatusType add_team(int teamId)

הקבוצה בעלת המזהה teamId נרשמה למירוץ, לכן יש לצרפה למערכת. לקבוצה אין ניסיון כלל. בעת ההכנסה אין מתמודדים בקבוצה.

פרמטרים:

teamId מזהה הקבוצה החדשה.

ערך החזרה:

ALLOCATION_ERROR	במקרה של בעיה בהקצאה/שחרור זיכרון.
INVALID_INPUT	אם $teamId \leq 0$.
FAILURE	אם teamId הוא מזהה של קבוצה לא מודחת.
SUCCESS	במקרה של הצלחה.

סיבוכיות זמן: $O(\log k)$ במקרה הגרוע, כאשר k הוא מספר הקבוצות שלא הודחו.



StatusType remove_team(int teamId)

הקבוצה בעלת המזהה teamId מודחת, ולכן צריך להוציאה מהמערכת. לאחר המחיקה, יתכן שתצורף למבנה קבוצה אחרת בעלת אותו מזהה.

פרמטרים:



teamId

מזהה הקבוצה.

ערך החזרה:

ALLOCATION_ERROR במקרה של בעיה בהקצאה/שחרור זיכרון.

INVALID_INPUT אם $teamId \leq 0$.

FAILURE אם אין קבוצה לא מודחת בעלת המזהה הנתון.

SUCCESS במקרה של הצלחה.

סיבוכיות זמן: $O(\log k)$ במקרה הגרוע, כאשר k הוא מספר הקבוצות הלא מודחות במערכת.

StatusType add_contestant (int contestantId, int teamId, const Skill &skill, int motivation, int missionsHad):

המתמודד בעל המזהה contestantId מצטרף לקבוצה בעלת המזהה teamId. המיומנות שלו היא skill, והמוטיבציה שלו היא motivation. בנוסף נתון שהמתמודד השתתף ב-missionsHad משימות.

פרמטרים:

contestantId

מזהה המתמודד שצריך להוסיף.

teamId

מזהה הקבוצה של המתמודד.

skill

המיומנות של המתמודד.

motivation

מספר שלם המתאר את המוטיבציה של המתמודד.

missionsHad

מספר המשימות בהן השתתף המתמודד.

ערך החזרה:

ALLOCATION_ERROR במקרה של בעיה בהקצאה/שחרור זיכרון.

INVALID_INPUT אם $contestantId \leq 0$ או $teamId \leq 0$ או skill לא תקין (wet2util.h) או $missionsHad < 0$ או $motivation < 0$.

FAILURE אם קיים כבר מתמודד עם מזהה contestantId, או שהיה קיים בעבר מתמודד עם מזהה contestantId, או שהקבוצה עם המזהה teamId לא קיימת במערכת.

SUCCESS במקרה של הצלחה.

סיבוכיות זמן: $O(\log k)$ משוערך, בממוצע על הקלט, כאשר k הוא מספר הקבוצות שלא הודחו.

output_t<int> duel(int teamId1, int teamId2)

שתי הקבוצות בעלות המזהים teamId1 ו-teamId2 הגיעו למשימת דו קרב. נסמן את הניסיון של הקבוצה ב teamExp. ניקוד המשימה האפקטיבי של קבוצה מוגדר לפי:

$$teamExp + \sum_{contestant \in team} contestant\ motivation$$

אם לקבוצה אחת ניקוד משימה אפקטיבי גדול יותר, הקבוצה בעלת הערך הגדול יותר תנצח. אם שני הערכים שווים, משווים את המיומנויות המשותפות של הקבוצות. הקבוצה בעלת יתרון במיומנות תנצח ורק אם לשתי הקבוצות יש מיומנות משותפת זהה, המשימה תסתיים בתיקו. **ניצחון** מגדיל את teamExp של הקבוצה המנצחת ב-3, **ותיקו** מגדיל ערך זה ב-1 לשתי הקבוצות.

מספר המשימות שכל אחד מהמתמודדים משני הצוותים השתתף בהן גדל ב-1.

פרמטרים:

מזהה הקבוצה הראשונה.	teamId1
מזהה הקבוצה השנייה.	teamId2
<u>ערך החזרה:</u> 0 בתיקו; 1 אם הקבוצה הראשונה ניצחה בגלל ניקוד משימה אפקטיבי; 2 אם הקבוצה הראשונה ניצחה בגלל יתרון במיומנות; 3 אם הקבוצה השנייה ניצחה בגלל ניקוד משימה אפקטיבי; 4 אם הקבוצה השנייה ניצחה בגלל יתרון במיומנות. ובנוסף סטטוס:	
במקרה של בעיה בהקצאה/שחרור זיכרון.	ALLOCATION_ERROR
אם $teamId2 \leq 0$ או $teamId1 = teamId2$.	INVALID_INPUT
אם אין קבוצה לא מודחת עם מזהה $teamId1$ או $teamId2$, או אחת הקבוצות ריקה.	FAILURE
במקרה של הצלחה.	SUCCESS
<u>סיבוכיות זמן:</u> $O(\log k)$ במקרה הגרוע, כאשר k מספר הקבוצות שלא הודחו.	

`output_t<int> get_contestant_missions_number (int contestantId)`

יש להחזיר את מספר המשימות הכולל בהן המתמודד בעל המזהה `contestantId` השתתף, גם אם הוא הודח.

פרמטרים:

מזהה המתמודד.	contestantId
<u>ערך החזרה:</u> מספר המשימות הכולל בהן השתתף המתמודד, ובנוסף סטטוס:	
במקרה של בעיה בהקצאה/שחרור זיכרון.	ALLOCATION_ERROR
אם $contestantId \leq 0$.	INVALID_INPUT
אם מעולם לא היה מתמודד עם מזהה <code>contestantId</code> .	FAILURE
במקרה של הצלחה.	SUCCESS
<u>סיבוכיות זמן:</u> $O(\log^* n)$ משוערך, בממוצע על הקלט, כאשר n מספר המתמודדים שהוכנסו למערכת מעולם.	

`output_t<int> get_team_experience(int teamId)`



יש להחזיר את הניסיון של הקבוצה בעלת המזהה `teamId`.

פרמטרים:

מזהה הקבוצה.	teamId
<u>ערך החזרה:</u> הניסיון של הקבוצה <code>teamId</code> , ובנוסף סטטוס:	
במקרה של בעיה בהקצאה/שחרור זיכרון.	ALLOCATION_ERROR
אם $teamId \leq 0$.	INVALID_INPUT
אם אין קבוצה לא מודחת במערכת עם מזהה <code>teamId</code> .	FAILURE
במקרה של הצלחה.	SUCCESS
<u>סיבוכיות זמן:</u> $O(\log k)$ במקרה הגרוע, כאשר k מספר הקבוצות הלא מודחות במערכת.	

`output_t<int> get_ith_collective_motivation_team(int i)`

נניח שמסדרים את קבוצות המתמודדים במערכת לפי המוטיבציה המשותפת שלהם, כלומר לפי:

$$\sum_{contestant \in Team} contestant\ motivation$$

שובר שוויון עבור קבוצות עם אותה מוטיבציה משותפת יהיה לפי `teamId` בסדר עולה.

יש להחזיר את ה-id של הקבוצה במיקום ה- i לפי הסדר הזה (אינדקס ראשון = 1).

פרמטרים:

- i אינדקס הקבוצה המבוקשת לפי הסדר שהוגדר.

ערך החזרה: מזהה של הקבוצה, ובנוסף סטטוס:

ALLOCATION_ERROR במקרה של בעיה בהקצאה/שחרור זיכרון.
 FAILURE אם אין קבוצות לא מודחות במערכת או אם $i < 1$ או $i > \text{number-of-teams}$.
 SUCCESS במקרה של הצלחה.
 סיבוכיות זמן: $O(\log k)$ במקרה הגרוע, כאשר k הוא מספר הקבוצות במערכת שלא הודחו.

`output_t <Skill> get_partial_team_skill(int contestantId)`

רוצים לחשב את המיומנות המשותפת של הקבוצה (רק אם לא הודחה) בה חבר המתמודד עם המזהה `contestantId` באופן חלקי בלבד. המתמודדים שנלקחים בחשבון בחישוב הם המתמודד בעל המזהה `contestantId` והמתמודדים שהצטרפו לפניו (לפי הסדר הכרונולוגי), ולא כל המתמודדים בקבוצה שלו. אם המתמודד בעל המזהה `contestantId` הוא המתמודד ה- i לפי הסדר הכרונולוגי של הקבוצה שלו, יש לחשב את:

$$\prod_{j=1}^i \text{skill of contestant } j \text{ in the team of } \text{contestantId}$$

שימו לב: יש להחזיר את אובייקט המיומנות עצמו שמתקבל מהמכפלה.

פרמטרים:

<code>contestantId</code>	מזהה המתמודד.
<u>ערך החזרה:</u> המיומנות המבוקשת, ובנוסף סטטוס:	
ALLOCATION_ERROR	במקרה של בעיה בהקצאה/שחרור זיכרון.
INVALID_INPUT	אם $\text{contestantId} \leq 0$.
FAILURE	אם מעולם לא היה מתמודד בעל מזהה <code>contestantId</code> . או המתמודד הודח.
SUCCESS	במקרה של הצלחה.

סיבוכיות זמן: $O(\log^* n)$ משוערך, בממוצע על הקלט.

`StatusType recruit(int recruitingTeamId, int recruitedTeamId)`

הקבוצה בעלת מזהה `recruitingTeamId` רוצה לגייס את הקבוצה בעלת מזהה `recruitedTeamId`. קבוצה תצליח לגייס קבוצה אחרת אם הסכום של הניסיון, המוטיבציה המשותפת והמיומנות המשותפת האפקטיבית שלו (השתמשו בפונקציה `getEffectiveSkill`) גדול ממשל האחר (נתייחס לקבוצה ריקה כאילו לעולם לא יכולה לגייס, ותמיד יכולה להיות מגוייסת על ידי קבוצה שאינה ריקה). במפורש, במצב בו זה אפשרי, קבוצה A יכולה לגייס את B אליה אם:

$$A's \text{ TeamExp} + A's \text{ totalMotivation} + A's \text{ effectiveSkill} > B's \text{ TeamExp} + B's \text{ totalMotivation} + B's \text{ effectiveSkill}$$

המזהה של הקבוצה הממוזגת נשאר זהה למזהה של הקבוצה המגייסת. הניסיון של הקבוצה הממוזגת יהיה סכום של הניסיון של שתי הקבוצות המקוריות. לכל מתמודד בקבוצה הממוזגת, מספר המשימות אותן ביצע נשמר.

פרמטרים:

<code>recruitingTeamId</code>	מזהה קבוצה מגייסת.
<code>recruitedTeamId</code>	מזהה קבוצה מגויסת.
<u>ערך החזרה:</u>	
ALLOCATION_ERROR	במקרה של בעיה בהקצאה/שחרור זיכרון.
INVALID_INPUT	אם אחד ה-id-ים אינו חיובי או שהם שווים.
FAILURE	אם אין קבוצות עם ה-id הנתונים, או שלא ניתן לבצע את המיזוג כי הקבוצה המגייסת ריקה או כי אי השוויון החזק אינו מתקיים.
SUCCESS	במקרה של הצלחה.

סיבוכיות זמן: $O(\log^* n + \log k)$ משוערך, כאשר k הוא מספר הקבוצות הלא מודחות במערכת ו- n הוא מספר המתמודדים שהיו אי פעם במערכת.

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 2 – מעודכן לתאריך 21.06.2026
עמוד 7 מתוך 10



input		output		דוגמה הרצה:
1	addTeam 1	1	addTeam: SUCCESS	
2	addTeam 2	2	addTeam: SUCCESS	
3	addTeam 3	3	addTeam: SUCCESS	
4	addContestant 10 1 2 1 1 1 50 0	4	addContestant: SUCCESS	
5	addContestant 11 1 3 1 0 1 30 2	5	addContestant: SUCCESS	
6	addContestant 12 2 1 2 0 1 40 1	6	addContestant: SUCCESS	
7	addContestant 13 3 2 0 0 3 10 0	7	addContestant: SUCCESS	
8	duel 1 2	8	duel: SUCCESS, 1	
9	getContestantMissionsNumber 10	9	getContestantMissionsNumber: SUCCESS, 1	
10	getContestantMissionsNumber 11	10	getContestantMissionsNumber: SUCCESS, 3	
11	getPartialTeamSkill 11	11	getPartialTeamSkill: SUCCESS, [6,3;3,2]	
12	getTeamExperience 1	12	getTeamExperience: SUCCESS, 3	
13	getIthCollectiveMotivationTeam 1	13	getIthCollectiveMotivationTeam: SUCCESS, 3	
14	recruit 1 3	14	recruit: SUCCESS	
15	getIthCollectiveMotivationTeam 1	15	getIthCollectiveMotivationTeam: SUCCESS, 2	
16	getPartialTeamSkill 13	16	getPartialTeamSkill: SUCCESS, [12,9;6,6]	
17	removeTeam 3	17	removeTeam: FAILURE	
18	addContestant 15 3 1 0 0 1 20 0	18	addContestant: FAILURE	
19	addTeam -5	19	addTeam: INVALID_INPUT	
20	duel 1 1	20	duel: INVALID_INPUT	



סיבוכיות מקום

סיבוכיות המקום הדרושה עבור מבנה הנתונים היא $O(n + k)$ במקרה הגרוע, כאשר n הוא מספר המתמודדים שנכנסו למערכת מההתחלה ו- k הוא מספר הקבוצות במערכת מההתחלה. כלומר לינארית בכמות פעולות ההוספה. סיבוכיות המקום הנדרשת עבור כל פעולה (כלומר, זיכרון "העזר" שכל פעולה משתמשת בו) אינה מצוינת לכל פעולה לחוד, אך אסור לעבור את סיבוכיות המקום הדרושה שמוגדרת לכל המבנה.

ערכי החזרה של הפונקציות:

כל אחת מהפונקציות מחזירה ערך מטיפוס `StatusType` שייקבע לפי הכלל הבא:

- תחילה, יוחזר `INVALID_INPUT` אם הקלט אינו תקין.
- אם לא הוחזר `INVALID_INPUT`:
- בכל שלב בפונקציה, אם קרתה שגיאת הקצאה/שחרור יש להחזיר `ALLOCATION_ERROR`. מצב זה אינו צפוי אלא באחד משני מקרים (לרוב): באמת השתמשתם בקלט גדול מאוד ולכן המבנה ניצל את כל הזיכרון במערכת, או שיש זליגת זיכרון בקוד.
- אם קרתה שגיאה אחרת, כפי שמצוין בכל פונקציה, יש להחזיר מיד `FAILURE` מבלי לשנות את מבנה הנתונים.
- אחרת, יוחזר `SUCCESS`.
- חלק מהפונקציות צריכות להחזיר בנוסף עוד פרמטר (`int` או `Skill`), לכן הן מחזירות אובייקט מטיפוס `output_t<T>`. אובייקט זה מכיל שני שדות: הסטטוס (`__status`) ושדה נוסף (`__ans`) מסוג `T`.
- במקרה של הצלחה (`SUCCESS`), השדה הנוסף יכיל את ערך החזרה, והסטטוס יכיל את `SUCCESS`. בכל מקרה אחר, הסטטוס יכיל את סוג השגיאה והשדה הנוסף לא מעניין.
- שני הטיפוסים (`output_t<T>`, `StatusType`) ממומשים כבר בקובץ "`wet2util.h`" שניתן לכם כחלק מהתרגיל.



הנחיות ודגשים כלליים:

חלק יבש:

- החלק היבש הוא חלק מהציון על התרגיל כפי שמצוין בנהלי הקורס.
- לפני מימוש הפעולות בקוד יש לתכנן היטב את מבני הנתונים והאלגוריתמים ולוודא כי באפשרותכם לממש את הפעולות בדרישות הזמן והזיכרון שלעיל.
- החלק היבש חייב להיות מוקלד.
- הגשת החלק הרטוב מהווה תנאי הכרחי לקבלת ציון על החלק היבש, כלומר, הגשה בה יתקבל אך ורק חלק יבש תגרוור ציון 0 על התרגיל כולו.
- יש להכין מסמך הכולל תיאור של מבני הנתונים והאלגוריתמים בהם השתמשתם בצירוף הוכחת סיבוכיות הזמן והמקום שלהם. חלק זה עומד בפני עצמו וצריך להיות מובן לקורא גם לפני העיון בקוד. אין צורך לתאר את הקוד ברמת המשתנים, הפונקציות והמחלקות, אלא ברמה העקרונית. חלק יבש איננו תיעוד קוד.
- 1. ראשית הציגו את מבני הנתונים בהם השתמשתם. **רצוי ומומלץ להיעזר בציור.**
- 2. לאחר מכן הסבירו כיצד מימשתם כל אחת מהפעולות הנדרשות. הוכיחו את דרישות סיבוכיות הזמן של כל פעולה תוך כדי התייחסות לשינויים שהפעולות גורמות במבני הנתונים.
- 3. הוכיחו שמבנה הנתונים וכל הפעולות עומדים בדרישת סיבוכיות המקום.
- החסמים הנתונים בתרגיל הם לא בהכרח הדוקים ולכן יכול להיות שקיים פתרון בסיבוכיות טובה יותר. מספיק להוכיח את החסמים הדרושים בתרגיל.
- רמת פירוט: יש להסביר את כל הפרטים שאינם טריוויאליים ושחשובים לצורך מימוש הפעולות ועמידה בדרישות הסיבוכיות. יש להסביר שינויים או תוספות בפעולות סטנדרטיות של מבנה נתונים שלמדנו אם נעשו כאלה. אין לדון בפרטים טריוויאליים (הפעילו את שיקול דעתכם בקשר לזה, ושאלו את האחראי על התרגיל אם אינכם בטוחים). **אין לצטט קטעים מהקוד כתחליף להסבר.** אין צורך לפרט אלגוריתמים שנלמדו בכיתה. כמו כן, אין צורך להוכיח תוצאות ידועות שנלמדו בכיתה, אלא מספיק לציין בבירור לאיזו תוצאה אתם מתכוונים. אין (וגם אין צורך) להשתמש בתוצאות של עצי דרגות והלאה.
- **על חלק זה לא לחרוג מ-8 עמודים.**
- והכי חשוב **keep it simple!**

חלק רטוב:

- מומלץ לממש תחילה את מבני הנתונים בצורה הכללית ביותר ורק אז לממש את הפונקציות הנדרשות בתרגיל.
- אנו ממליצים בחום על מימוש **Object Oriented**, **C++**, מימוש כזה יאפשר לכם להגיע לפתרון פשוט וקצר יותר לפונקציות אותן עליכם לממש ויאפשר לכם להכליל בקלות את מבני הנתונים שלכם (זכרו שיש תרגיל רטוב נוסף בהמשך הסמסטר).
- פקודת הקימפול שמורצת ב-gradescope הינה: `g++ -std=c++14 -DNDEBUG -Wall -o main.out *.cpp`
- חתימות הפונקציות שעליכם לממש ומספר הגדרות נמצאים בקובץ `Racenion26b2.h`.
- אין לשנות את הקבצים `main26b2.cpp` ו-`wet2util.h` אשר סופקו כחלק מהתרגיל, ואין להגיש אותם. ישנה בדיקה אוטומטית שאין בקוד שימוש ב-STL, ובדיקה זו נופלת אם מגישים גם את `main26b2.cpp`.
- את שאר הקבצים ניתן לשנות, ותוכלו להוסיף קבצים נוספים כרצונכם, ולהגיש אותם.
- העיקר שהקוד שאתם מגישים יתקמפל עם הפקודה לעיל, כאשר מוסיפים לו את שני הקבצים `main26b2.cpp` ו-`wet2util.h`.
- עליכם לממש בעצמכם את כל מבני הנתונים (**אין להשתמש במבנים של STL** ואין להעתיק מבני נתונים מהאינטרנט). **כחלק מתהליך הבדיקה אנו נבצע בדיקה ידנית של הקוד ונוודא שאכן מימשתם את מבני הנתונים שבהם השתמשתם.**
- בפרט, אסור להשתמש ב-`std::pair`, `std::vector`, `std::iterator`, או כל אלגוריתם של STL, רשימה מלאה של הספריות להן אסור לעשות include נמצאת בקובץ `dont_include.txt`.
- **ניתן** להשתמש במצביעים חכמים (כמו `shared_ptr`), בספריית `math` או בספריית `exception`.
- חשוב לוודא שאתם מקצים/משחררים זיכרון בצורה נכונה (מומלץ לוודא עם `valgrind`). לא חייבים לעבוד עם מצביעים חכמים, אך אם אתם מחליטים כן לעשות זאת, וודאו שאתם משתמשים בהם נכון. (תזכרו שהם לא פתרון קסם, למשל, כאשר יוצרים מעגל בהצבעות). שימו לב שהעתקת מצביעים חכמים היא איטית מאד ולכן יכולה לגרום ל-timeout. עדיף להעביר `shared_ptr` כשרוצים להעביר `by reference`.
- שגיאות של `ALLOCATION_ERROR` בד"כ מעידות על זליגה בזיכרון.
- על הקוד להתקמפל ולעבור את כל הבדיקות שמפורסמות לכם ב-gradescope. הטסטים שמורצים באתר מייצגים את הבדיקות אותן נריץ בנתינת הציון, כאשר פרסמנו 5 מתוך 50.

מבני נתונים 234218 אביב תשפ"ו

גיליון רטוב מספר 2 – מעודכן לתאריך 21.06.2026
עמוד 10 מתוך 10



- אותם טסטים שבgradescope גם מפורסמים כקבצי קלט ופלט, יחד עם סקריפט בשם run_tests.py שנכתב בשביל python 3.6 ומעלה, המאפשר לבדוק את הקוד שלכם. מומלץ לבדוק את התרגיל לוקאלית לפני שמגישים. צורף קובץ readme.txt למקרה שלא ברור איך להריץ את הטסטים.
- במידה ויש timeout על אחד מהטסטים זה יחשב ככישלון בטסט. עליכם לכתוב קוד יעיל במידת הסביר. אין לדאוג מכך יותר מידי, הרוב המוחלט של הפתרונות שעומד בסיבוכיות עומד גם בזמני הריצה.
- **שימו לב:** התוכנית שלכם תיבדק על קלטים רבים ושונים מקבצי הדוגמא הנ"ל. יחד עם זאת, הטסטים האלו מייצגים מבחינת אורך ואופן היצירה שלהם את השאר.

אופן ההגשה:

הגשת התרגיל הנה דרך [אתר ה-gradescope של הקורס](#).

החלק הרטוב:

יש להגיש רק את קבצי הקוד שלכם (לרוב קבצי .cpp, .h, בלבד ואפשר גם להגיש אותם כקובץ zip).

החלק היבש:

יש להגיש קובץ PDF אשר מכיל את הפתרון היבש. החלק היבש חייב להיות מוקלד.

■ שימו לב כי אתם מגישים את שני החלקים הנ"ל, במטלות השונות.

- לאחר ההגשה, יש באפשרותכם לבצע שינויים ולהגיש שוב. ההגשה האחרונה היא זו שתחשב.
- הערכת הציון שמופיעה ב-gradescope אינה הציון הסופי על המטלה. הציון הסופי יתפרסם רק לאחר ההגשות המאוחרות של משרתי המילואים.
- במידה ואתם חושבים שישנה תקלה מהותית במערכת הבדיקה ב-gradescope נא להעלות זאת בפורום הפיאצה ונטפל בה בהקדם.

דחיות ואיחורים בהגשה:

- דחיות בתרגיל הבית תינתנה אך ורק לפי תקנון הקורס.
- 5 נקודות יורדו על כל יום איחור בהגשה ללא אישור מראש. באפשרותכם להגיש תרגיל באיחור של עד 5 ימים ללא אישור. תרגיל שיוגש באיחור של יותר מ-5 ימים ללא אישור מראש יקבל 0.
- במקרה של איחור בהגשת התרגיל יש עדיין להגיש את התרגיל אלקטרונית דרך אתר הקורס.
- בקשות להגשה מאוחרת יש להפנות רק למתרגל האחראי: goldstein@campus.technion.ac.il.
- לאחר קבלת אישור במייל על הבקשה, מספר הימים שאושרו לכם נשמר אצלנו. לכן, אין צורך לצרף להגשת התרגיל אישורים נוספים או את שער ההגשה באיחור.

בהצלחה!